Brickts SQL course — Week 8 — Use RDF and SPARQL practically

sparql-playground / lessons/sql

Generated for offline / Kindle reading

Week 8 — Use RDF and SPARQL practically

This pack concatenates the six daily lessons and the weekly challenge, then the Week primary-reading links from SOURCES.md. Work the SQL in `psql` against `brickts_course`; keep notes under ignored `student-work/sql/`.

Contents

- Day 50: Day 50 — Read the small amount of RDF your SQL workflow needs
- Day 51: Day 51 — Discover typed equipment with a small SELECT

- Day 52: Day 52 — Follow point ownership through an equipment part tree
- Day 53: Day 53 — Resolve a point reference before a bounded SQL read
- Day 54: Day 54 — Build an explicit Brick-to-rule adapter
- Day 55: Day 55 — Review graph changes and inspect the read-only web boundary
- Day 56 (weekly challenge): Day 56 — Week 8 independent challenge — a model-driven SQL analytics report

---

Day 50 — Read the small amount of RDF your SQL workflow needs

Previous: Day 49 · **Week 8 · 45–75 minutes.** Prerequisites: Days 1–49, or equivalent SQL skills.

Goal

Recognize triples, IRIs, prefixes and literals in the course metadata graph.

Before you start

Open `lessons/sql/model.ttl` in an editor. It is a small synthetic instance graph, separate from stored samples. Run `uv run python lessons/sql/graph_lab.py --check` from the repository root; this checks fixture bindings offline rather than querying a running database.

Concept

An RDF triple has subject, predicate and object. The subject and predicate are identifiers; the object may be another identifier or a literal value. An IRI identifies a thing or relationship. A prefix abbreviates an IRI: `brick:` expands to `https://brickschema.org/schema/Brick#`. Prefix spelling is local shorthand, not identity.

Turtle's `a` abbreviates `rdf:type`. A quoted label such as `"Supply A"` is a literal, not an equipment identifier. The two sites deliberately share that equipment label. Course IRIs use `https://example.org/course#`; their UUID-bearing suffixes are opaque. Do not split suffixes to recover database keys. Follow explicit reference properties instead.

The graph describes which point means what and where its history is referenced. It does not contain the temperature and pressure observations from the SQL sample table. This week teaches practical discovery; you do not need to author a new ontology.

Tiny example

```
SELECT ?subject ?predicate ?object
WHERE { ?subject ?predicate ?object }
LIMIT 6
```

Run the supplied equivalent with `uv run python lessons/sql/graph_lab.py lessons/sql/sparql/day50_triples.rq`. JSON bindings preserve RDF term kinds such as `uri`, `literal` and `bnode`. A blank-node identifier is local to that graph serialization, not a permanent timeseries key.

SQL challenge

Choose one equipment and one point from the Turtle file. Write a short mapping between their RDF facts and columns in `course.equipment` and `course.point`. Verify identities through explicit metadata fields, distinguishing a label literal from an IRI and a reference node from a stored sample.

Experiment

Copy the tiny query into your own file and rename a prefix while keeping its expanded namespace unchanged. Predict whether results change. Then change the namespace itself and explain why matching differs.

Acceptance checks

- The offline check verifies 42 point bindings.
- You can label subject, predicate and object in three actual triples.
- You distinguish an IRI, literal and blank node in JSON.
- No history is claimed to have been read merely because metadata was returned.

Stretch and reflection

Which metadata could an application cache, and what change would require invalidating that cache? Save the question for the revision discussion on Day 55.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day50/` directory. Label evidence

observed, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 49 ·

Day 51 — Discover typed equipment with a small SELECT

Previous: Day 50 · **Week 8 · 45–75 minutes.** Prerequisites: Days 1–50, or equivalent SQL skills.

Goal

Find equipment by an explicit Brick class and preserve its identity across the graph-to-SQL boundary.

Before you start

Run `uv run python lessons/sql/graph_lab.py lessons/sql/sparql/day51_ahus.rq`. The helper loads only the course instance graph. It does not load the Brick ontology or infer class hierarchies.

Concept

A basic SPARQL pattern resembles a join: repeated variables require matching RDF terms. `?equipment a brick:Air_Handler_Unit`

matches that explicitly asserted class. Add label and site patterns to retrieve context, then use `SELECT` to choose output columns. `ORDER BY` makes a result easier to compare; it does not create a meaningful identity.

An exact class match is not the same as asking for every subclass of a broad class. The course graph has no `rdfs:subClassOf` closure, so a broad `brick:Equipment` query may return nothing even when explicit AHUs exist. Applications that depend on subclass discovery must explicitly load/query the necessary ontology or inference behavior. This course baseline keeps that boundary visible.

Tiny example

```
PREFIX brick:
<https://brickschema.org/schema/Brick#>
SELECT ?equipment
WHERE { ?equipment a
brick:Air_Handler_Unit }
ORDER BY ?equipment
LIMIT 10
```

The supplied query additionally returns site, label and the explicit equipment UUID literal. It is a discovery example, not a report of measured operation.

SQL challenge

Write your own graph inventory query returning equipment IRI, explicit UUID field, site and label. Compare its rows with a SQL inventory over `course.equipment`. Demonstrate that labels alone cannot join the two inventories safely. Keep IRI and UUID fields separately named in your evidence.

Experiment

Replace the class in your copied query with `brick:Equipment`. Predict the instance-only result before running. Explain why an empty exact-class result is not proof that the building has no equipment. Restore the explicit AHU class.

Acceptance checks

- AHU discovery returns three rows: two north, one south.
- Supply A appears twice with different site and equipment identities.
- Graph and SQL inventories agree through explicit UUID fields.
- Your note states that subclass inference is absent in this helper.

Stretch and reflection

What is the minimum additional metadata needed to query all fan subtypes reliably? Describe the requirement without expanding this lab into an ontology-authoring project.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day51/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 50 ·

Day 52 — Follow point ownership through an equipment part tree

Previous: Day 51 · **Week 8 · 45–75 minutes.** Prerequisites: Days 1–51, or equivalent SQL skills.

Goal

Discover an AHU's points whether ownership is direct or attached to a component part.

Before you start

Run `uv run python lessons/sql/graph_lab.py lessons/sql/sparql/day52_owned_points.rq`. The starter model places points directly on AHUs; its zero-length path case is therefore meaningful. It does not contain a fabricated fan subassembly.

Concept

`brick:isPointOf` links a point to its owner. Equipment can have parts: `brick:isPartOf` walks from child toward parent, while

`^brick:hasPart` traverses the inverse of a parent-to-child assertion. The path `(brick:isPartOf|^brick:hasPart)*` accepts either form and zero or more steps. Zero steps let the owner already be the requested AHU.

Do not equate ownership with airflow or spatial containment. A point's owner, a system that feeds a zone and a building containing equipment express different relationships. Keep the intermediate owner in discovery results when it helps explain why a point was selected. A future model with multiple paths may require explicit duplicate checks and sensor-selection policy.

Tiny example

```
PREFIX brick:
<https://brickschema.org/schema/Brick#>
SELECT ?point ?owner ?equipment
WHERE {
  ?equipment a brick:Air_Handler_Unit .
  ?point brick:isPointOf ?owner .
  ?owner (brick:isPartOf|^brick:hasPart)*
?equipment .
}
LIMIT 10
```

This returns metadata associations, not aligned timeseries observations. The supplied file also projects site and teaching role.

SQL challenge

Discover all points in each AHU's ownership tree. Compare discovered equipment/role pairs with `course.point`, preserving equipment identity and counting selected points per required role. Design the SQL-side audit that would reject a missing or ambiguous pressure-role selection before reading history.

Experiment

In your copied query, replace `*` with `+`, which requires at least one part-tree step. Predict the result on the direct-ownership fixture. Explain why this differs from simply joining `isPointOf` directly to an AHU when parts exist in a larger model.

Acceptance checks

- The complete ownership query returns 42 point associations, 14 per AHU.
- Direct ownership works because zero path steps are allowed.
- The `+` experiment has no matching AHU part path in this starter graph.
- Every discovered point retains its site, equipment and explicit role context.

Stretch and reflection

If two pressure sensors share a role, which selection information is missing? Explain why `LIMIT 1` would conceal that problem rather than resolve it.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day52/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 51 ·

Day 53 — Resolve a point reference before a bounded SQL read

Previous: Day 52 · **Week 8 · 45–75 minutes.** Prerequisites: Days 1–52, or equivalent SQL skills.

Goal

Follow the model's timeseries reference into a parameterized, bounded read without deriving keys from labels or IRIs.

Before you start

Run `uv run python lessons/sql/graph_lab.py lessons/sql/sparql/day53_power_refs.rq`. It returns power-point metadata for three AHUs. The database backend is `postgresql`; the connection property contains the environment-variable name `COURSE_DSN`, never its value.

Concept

The read path is point → `ref:hasExternalReference` → `ref:TimeseriesReference` → `ref:hasTimeseriesId` plus `ref:storedAt`. The database IRI selects the configured store adapter. The opaque timeseries ID selects history within that store. Validate exactly one reference, one ID and one database before reading.

For this teaching fixture only, the frozen timeseries ID equals SQL `point_id`. An adapter must declare that mapping; another historian may use a different opaque key. The IRI suffix, equipment label and original CSV column are not alternate runtime lookups. Keep site scope as well as point identity when reading `course.sample`.

Tiny example

```
PREPARE day53_read (text, uuid,
         timestamptz, timestamptz) AS
SELECT ts, value, quality
FROM course.sample
WHERE site_id = $1 AND point_id = $2
  AND ts >= $3 AND ts < $4
ORDER BY ts
LIMIT 12;
```

This prepares a statement with bound values; it does not execute a sample read. Use a discovered site/ID and explicit UTC boundaries when executing or calling a driver. Never concatenate graph text into SQL syntax.

SQL challenge

Resolve one discovered power point with `graph_lab.py --resolve POINT_IRI`, replacing `POINT_IRI` with its returned full IRI. Record the returned ID and database. Read a bounded hour using parameters, retain quality and compare the result with the same point selected through SQL metadata. Write the adapter contract connecting graph fields to SQL parameters.

Experiment

Specify a deliberately nonexistent opaque ID in a scratch read. Compare an empty history result with a graph reference that failed cardinality validation. Explain why those require different error messages.

Acceptance checks

- Discovery returns three power references with `unit:KiloW` and one configured

database.

- `--resolve` returns point, timeseries ID and database; it reads no samples.
- A one-hour, complete power window has twelve rows; Jan 5 15:00–16:00Z north Supply A includes one bad/null reading.
- History is bounded and values are bound separately from SQL text.

Stretch and reflection

Which component would change if history moved to a lakehouse while point identities remained stable? Explain how the metadata and store-adapter boundary supports that change.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day53/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 52 ·

Day 54 — Build an explicit Brick-to-rule adapter

Previous: Day 53 · **Week 8 · 45–75 minutes.** Prerequisites: Days 1–53, or equivalent SQL skills.

Goal

Separate graph semantics, course role names, units and production rule inputs before comparing FDD behavior.

Before you start

Read the Open-FDD bridge, including its pinned reference and distinction between this repository's Brick profile and Open-FDD's Haystack conventions and extensions. Run `uv run python lessons/sql/graph_lab.py lessons/sql/sparql/day54_role_units.rq`.

Concept

Brick classes describe equipment and point meaning using RDF relationships. Haystack represents semantic information through its own tags, definitions and relationships. Open-

FDD also has a specific rule-input contract. A role string such as `duct_static` is a local adapter key, not proof that every system uses that exact name or unit.

An adapter must map identity, signal meaning, unit, timestamps, quality and required input names. Pressure in Pa cannot be sent as an assumed value in another pressure unit. A numeric percentage cannot become a Boolean merely by renaming the column. A measured fan status is not a fan command. The rule catalog and pinned implementation determine which signal the production rule actually requires.

Tiny example

```
PREFIX brick:
<https://brickschema.org/schema/Brick#>
PREFIX ex: <https://example.org/course#>
SELECT DISTINCT ?role ?unit
WHERE { ?point ex:role ?role ;
brick:hasUnit ?unit }
ORDER BY ?role
```

This inventories fourteen local role/unit pairs. It does not establish production rule applicability or perform unit conversion.

SQL challenge

Write a scratch adapter table or documented mapping for pressure, pressure setpoint, fan status and fan-speed command. Include course role, Brick class, QUDT unit, candidate production input and required transformation or unresolved question. Use SQL metadata to audit presence and units before constructing any rule-ready result.

Experiment

Describe how your adapter rejects a scratch pressure value labelled kPa when Pa is expected. Then test a deliberate status-versus-command mismatch in scratch mapping metadata. A failed audit is the expected evidence; do not silently relabel the signal to make a rule run.

Acceptance checks

- Role/unit inventory has fourteen pairs, including Pa, kW, °C, percent and unitless status.
- Mapping retains status and command as distinct signals.
- Unsupported units or ambiguous source roles block adapter success with a reason.

- Your report calls the Week 7 predicate a teaching rule and identifies what must be checked before production comparison.

Stretch and reflection

Would two correctly mapped implementations necessarily return identical faults if their startup and confirmation policies differ? Explain which policy metadata belongs beside any comparison.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day54/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 53 ·

Day 55 — Review graph changes and inspect the read-only web boundary

Previous: Day 54 · **Week 8 · 45–75 minutes.** Prerequisites: Days 1–54, or equivalent SQL skills.

Goal

Understand graph CRUD and revision consistency while using only the implemented read surface.

Before you start

The offline course helper supports SELECT/ASK and reference resolution. The existing brickts HTTP API has its own building model and SQLite history; it is separate from the PostgreSQL course fixture. HTTP examples below are learner-run only when that local app is already prepared according to the repository guide.

Concept

CRUD means create, read, update and delete. For graph metadata, an update can change a point's owner, unit or external reference while its history remains stored elsewhere. A staged model change should validate reference cardinality, identity and store reachability before publication. A reader should resolve against one coherent model revision; caching a point mapping across a model replacement can read the wrong history.

These are design requirements, not promises that this course exposes mutation endpoints or revision tokens. The shipped SPARQL boundary is read-only. Record a captured model file and its checksum as evidence of the exact artifact you examined. A Turtle byte checksum identifies those bytes, not semantic equivalence across differently ordered serializations.

Tiny example

```
curl --fail
         http://127.0.0.1:8000/api/model/ttl?
         scope=model
curl --fail -X POST
         http://127.0.0.1:8000/api/sparql
         \
  -H 'Content-Type: application/json' \
  --data '{"query":"ASK { ?s ?p ?o }"}'
```

These inspect the app's graph. The SPARQL response wraps typed results in `results` plus `meta`; offline helper JSON is the direct SPARQL result. `GET /api/points/{point_id}/timeseries` accepts app-discovered point IDs, optional integer epoch-second `start`/`end`, and `limit`. Do not substitute course UUIDs into a different model.

SQL challenge

Write a proposed change-review record for moving a point to another database reference. Include old/new identity, model artifact, validation, cache invalidation and rollback criteria. Classify proposed CRUD operations by their effect on graph metadata versus stored history; do not perform them. Contrast graph discovery with the subsequent bounded SQL read.

Experiment

Compare an offline query result shape with the documented HTTP wrapper. If the app is unavailable, record the HTTP comparison as not run. Explain a stale-cache scenario using two paper model revisions.

Acceptance checks

- Your design prevents a reader from mixing half-applied reference changes.
- The read-only boundary and distinct datasets are explicit.
- No implemented graph-write API, revision token or production deployment is invented.
- Proposed deletion of metadata is not described as automatic deletion of history.

Stretch and reflection

What coordinated failure could occur between publishing a graph and provisioning a store key? Identify the validation that would prevent advertising an unreadable reference.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day55/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 54 ·

Day 56 — Week 8 independent challenge — a model-driven SQL analytics report

Previous: Day 55 · **Week 8 review · 2–4 hours, split across sessions as needed.** Prerequisites: Days 50–55, or equivalent skills.

Project brief

Build an independently written report whose equipment and point selection comes from the course RDF model, then whose calculations run in SQL over bounded history. Cover observed runtime, energy and one of comfort or the teaching pressure condition. A reviewer should be able to follow the path from semantic discovery to a number and back to evidence.

Acceptance criteria

- SPARQL discovers all three AHUs and required points by explicit class,

ownership and role; labels do not supply identity.
- Every selected point has one external reference, one opaque timeseries ID and one database. Role and unit audits reject missing or ambiguous inputs.
- SQL uses the adapter's explicit IDs, site scope, bound values and half-open reporting window. No IRI parsing or source-column lookup substitutes for reference resolution.
- Full-window observed runtime totals 48 equipment-hours and observed energy totals 467 kWh, with power coverage stated.
- Detail, summary and chosen additional metric retain their declared grain, units and policy.
- Model evidence identifies the captured artifact; the report distinguishes offline fixture validation from live API evidence.

Deliverables

Submit your SPARQL, SQL, adapter contract, compact results and a short architecture note. Include one negative case for a broken reference or incompatible unit, plus one numerical counterexample involving missing coverage. Keep any web/API inspection separate from the course database result.

Self-review

Could the store change without rewriting the analytics selection logic? Could a graph revision invalidate a cached mapping? Explain which validations happen before history reads and which happen before calculating the final metric.

Label results **observed**, **fixture-only**, or **not run**. This page intentionally provides no worked solution or implementation sequence. Attempt the challenge independently before consulting the week's references.

Previous: Day 55 ·

Primary reading — Week 8

Use the lesson first, then these references for precise behavior. Full list: SOURCES.md.

Week 8

- Brick core concepts
- Brick external timeseries references
- W3C SPARQL query language
- Typed SPARQL JSON results

- Haystack RDF mapping